

LangChain & LangGraph

LangChain과 LangGraph

01 LangChain

목차 01 LangChain

1. LLM을 활용한 시스템의 일반적인 아키텍처
2. LangChain 패키지 구조

1. LLM을 활용한 시스템의 일반적인 아키텍처

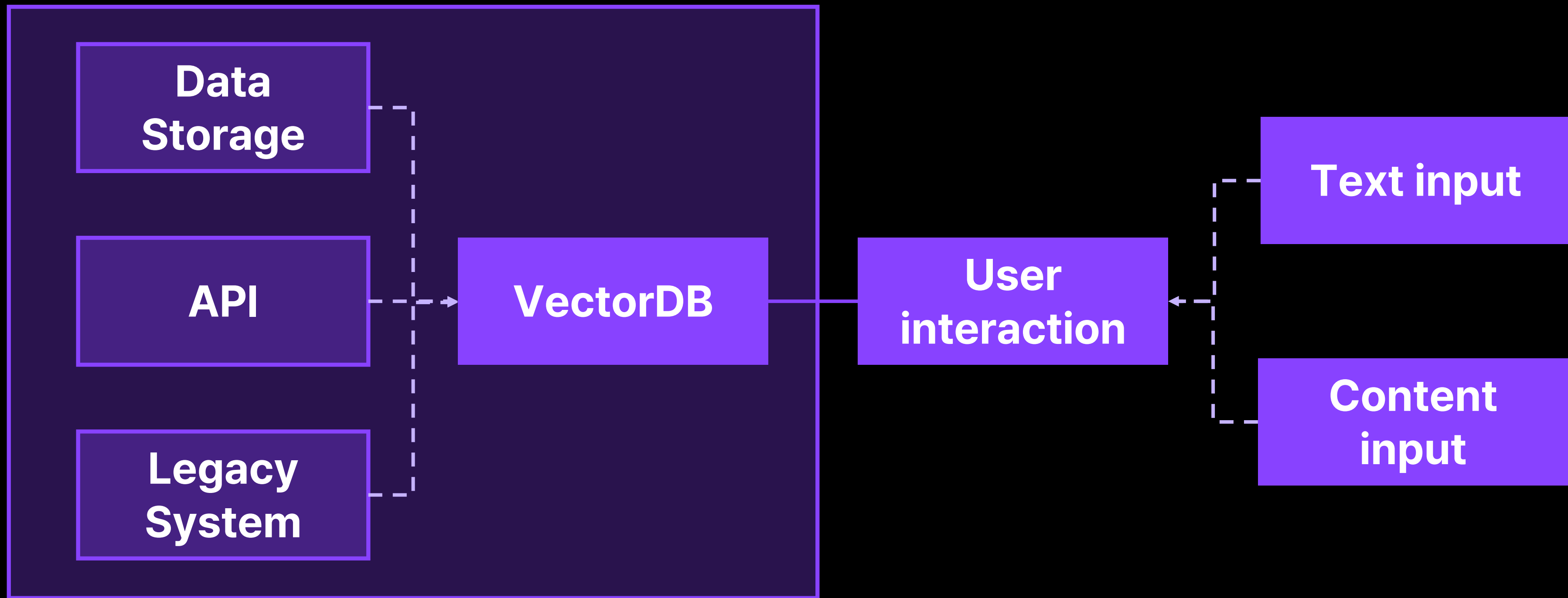


LLM을 활용한 시스템의 일반적인 아키텍처

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



데이터를 수집할 수 있는
data source와 연결

- 기존 legacy data
- API 또는 실시간 data

Data를 VectorDB에 저장

- Embedding vector
- Meta data

사용자 인풋 처리

- 시스템의 포맷에 맞는 형태로 재구성
- 시스템 아웃풋을, 사용자와 chat 형식으로 제공

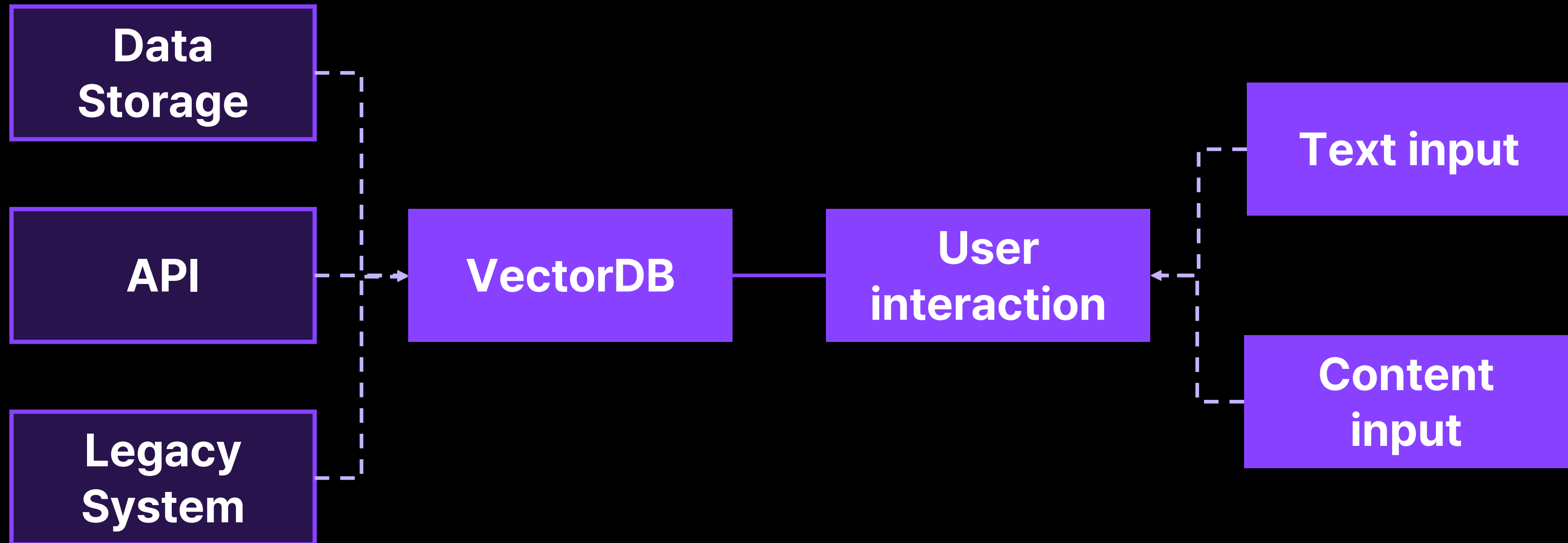


LLM을 활용한 시스템의 일반적인 아키텍처

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



데이터를 수집할 수 있는
data source와 연결

- 기존 legacy data
- API 또는 실시간 data

Data를 VectorDB에 저장

- Embedding vector
- Meta data

사용자 인풋 처리

- 시스템의 포맷에 맞는 형태로 재구성
- 시스템 아웃풋을, 사용자와 chat 형식으로 제공

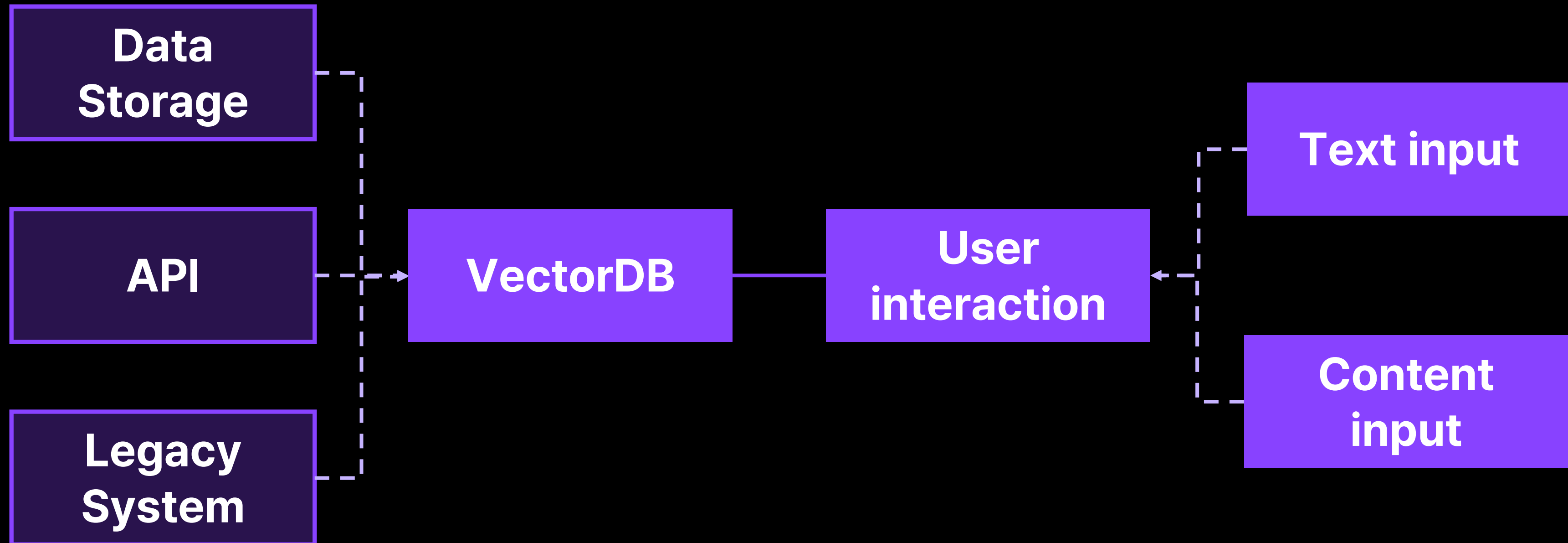


LLM을 활용한 시스템의 일반적인 아키텍처

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



데이터를 수집할 수 있는
data source와 연결

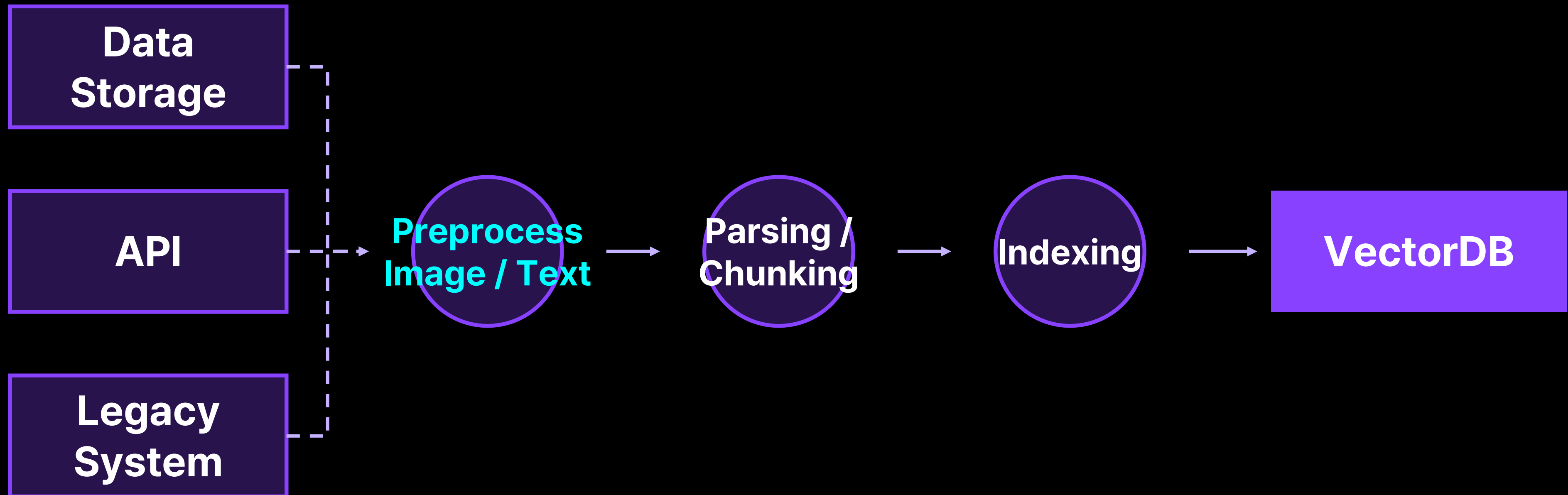
- 기존 legacy data
- API 또는 실시간 data

Data를 VectorDB에 저장

- Embedding vector
- Meta data

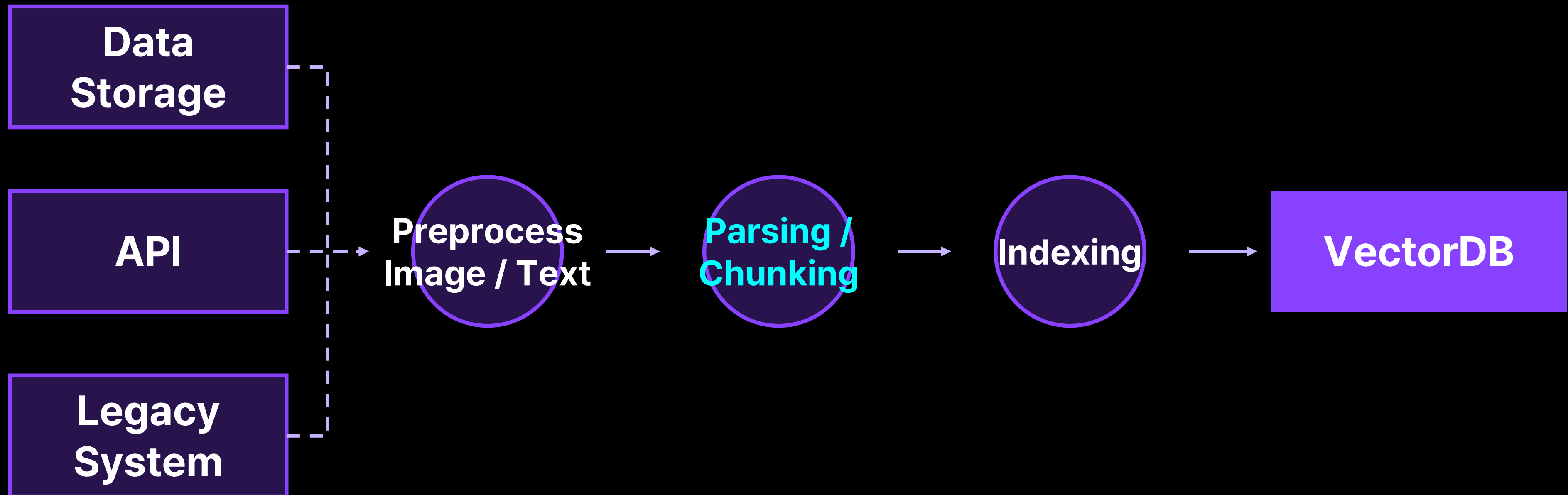
사용자 인풋 처리

- 시스템의 포맷에 맞는 형태로 재구성
- 시스템 아웃풋을, 사용자와 chat 형식으로 제공



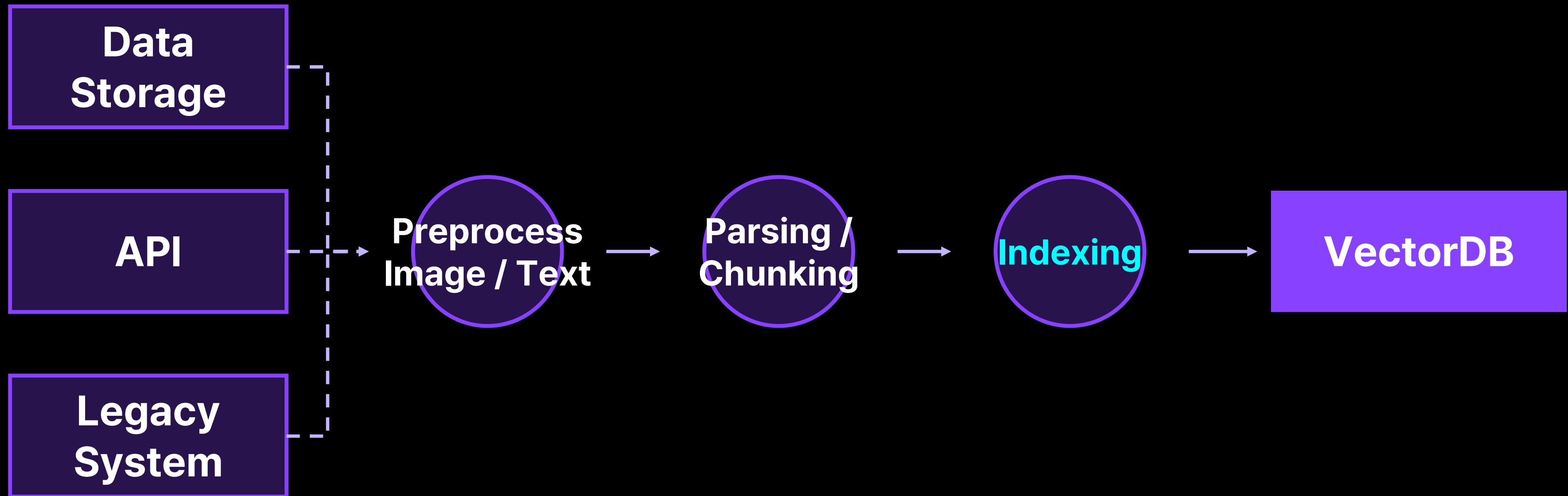
Preprocess Image / Text
:파일 전처리

- PDF, word 등 파일들을 텍스트로 변환
- 이미지의 경우 Standardization 등 표준화



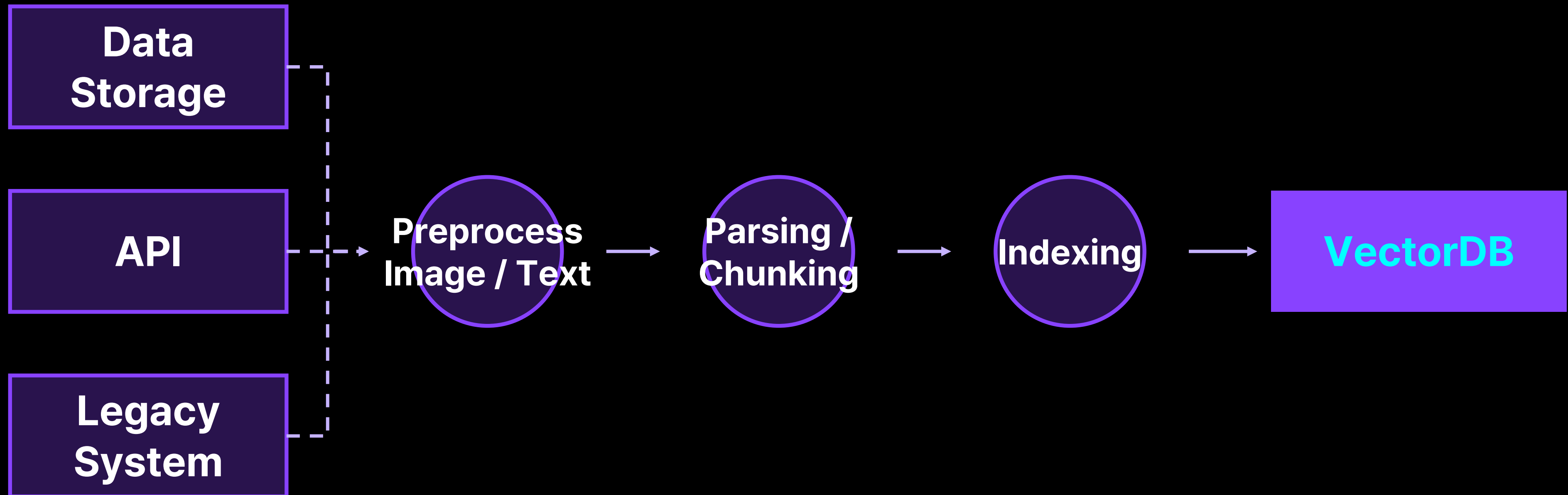
Parsing / Chunking
:텍스트 전처리

- 텍스트의 경우 다수의 chunk로 변환
- Semantic parser
- Character parser
- ...



Indexing
:VDB의 형태에 맞는 index의 형태로 변환

- Dense vector
- Sparse vector
- Metadata



Vector DataBase
:VDB 구축

- Qdrant
- ChromaDB
- ...

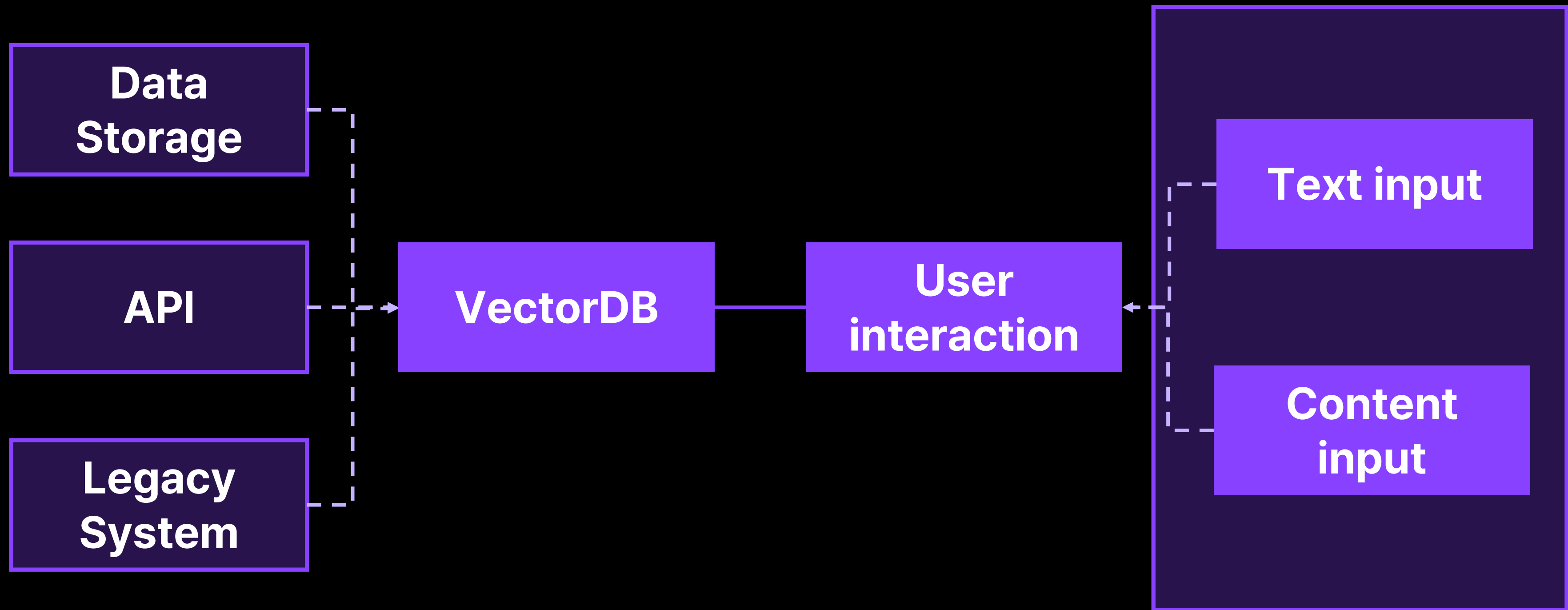


LLM을 활용한 시스템의 일반적인 아키텍처

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



데이터를 수집할 수 있는
data source와 연결

- 기존 legacy data
- API 또는 실시간 data

Data를 VectorDB에 저장

- Embedding vector
- Meta data

사용자 인풋 처리

- 시스템의 포맷에 맞는 형태로 재구성
- 시스템 아웃풋을, 사용자와 chat 형식으로 제공

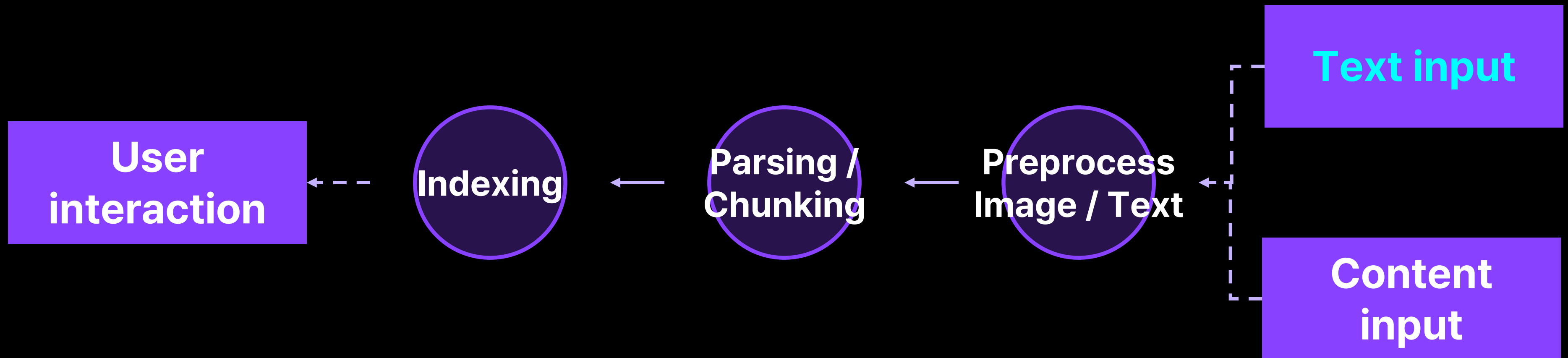


아키 zoom in - 사용자 input

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



User Input
:사용자 인풋의 형태

- Text input
 - 일반 텍스트 인풋

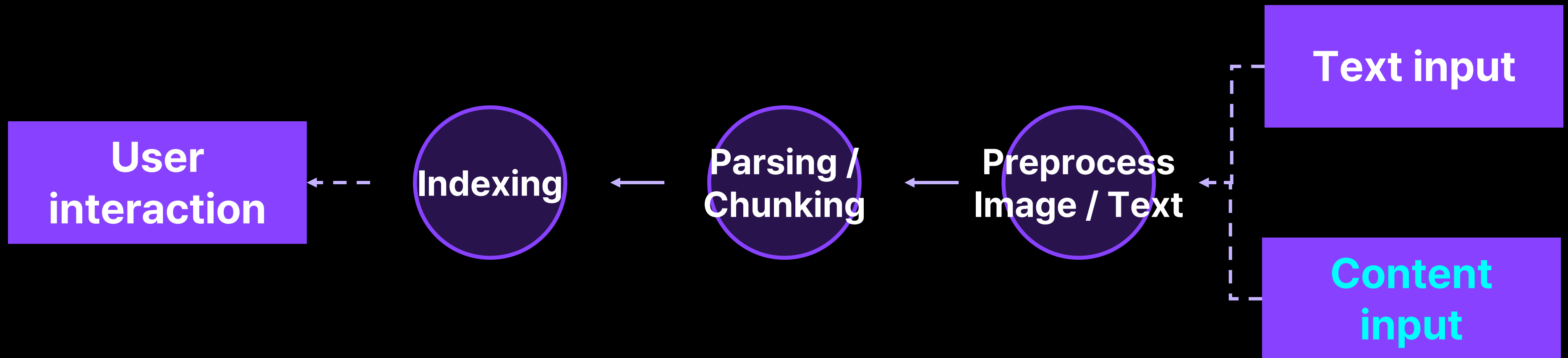


아키 zoom in - 사용자 input

LangChain과
LangGraph

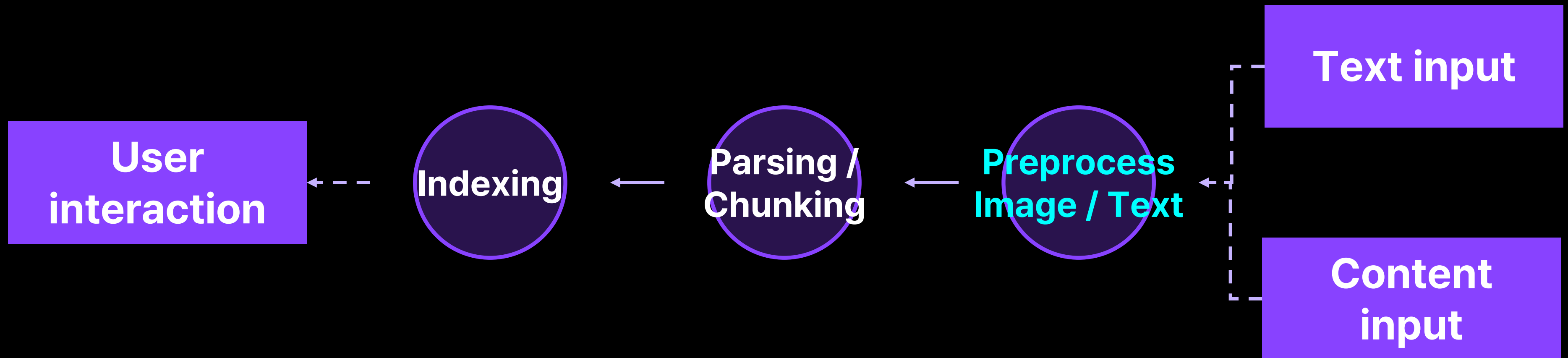
01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



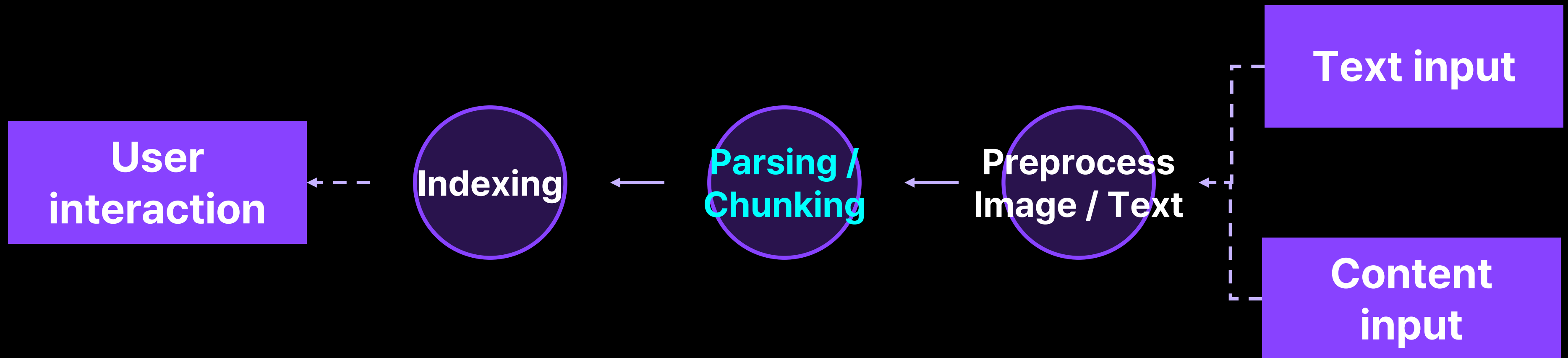
User Input
:사용자 인풋의 형태

- 다양한 형태의 인풋
 - 파일 / 일반 text / 텍스트 형태의 url, code 등



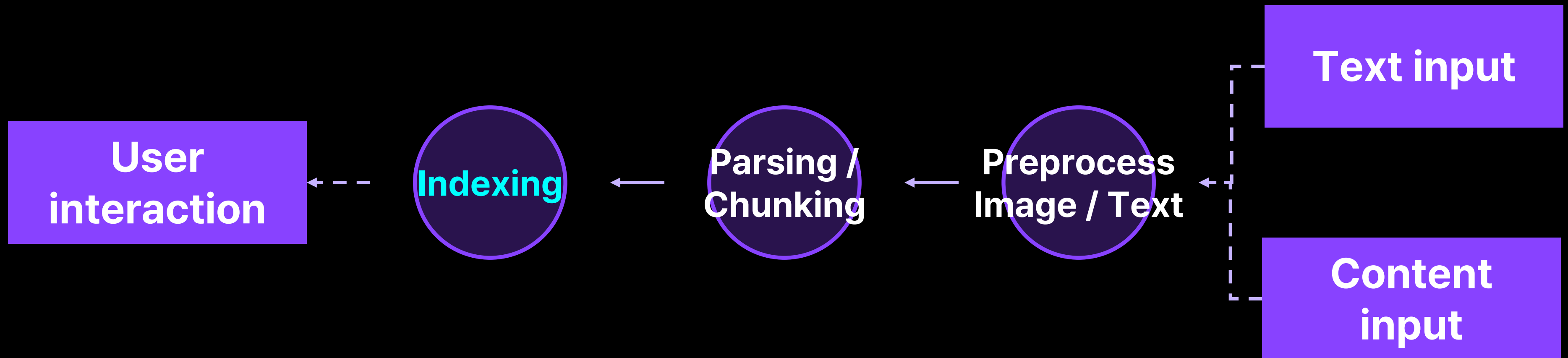
User input 처리
:VDB 구축과 유사한 형태의 프로세스

- 텍스트화 및 input 전처리
- 활용 가능한 형태로 표준화



User input 처리
:VDB 구축과 유사한 형태의 프로세스

- 구축된 system의 input 형태와 유사한 형태로 변환
 - Parsing / chunking의 경우 VDB에 저장한 information chunk와 유사한 형태 및 format으로 변환



User input 처리
:VDB 구축과 유사한 형태의 프로세스

- 구축된 system의 input 형태와 유사한 형태로 변환
 - Indexing과 같은 경우 동일한 embedding 모델 활용
 - Meta data filtering 등을 활용

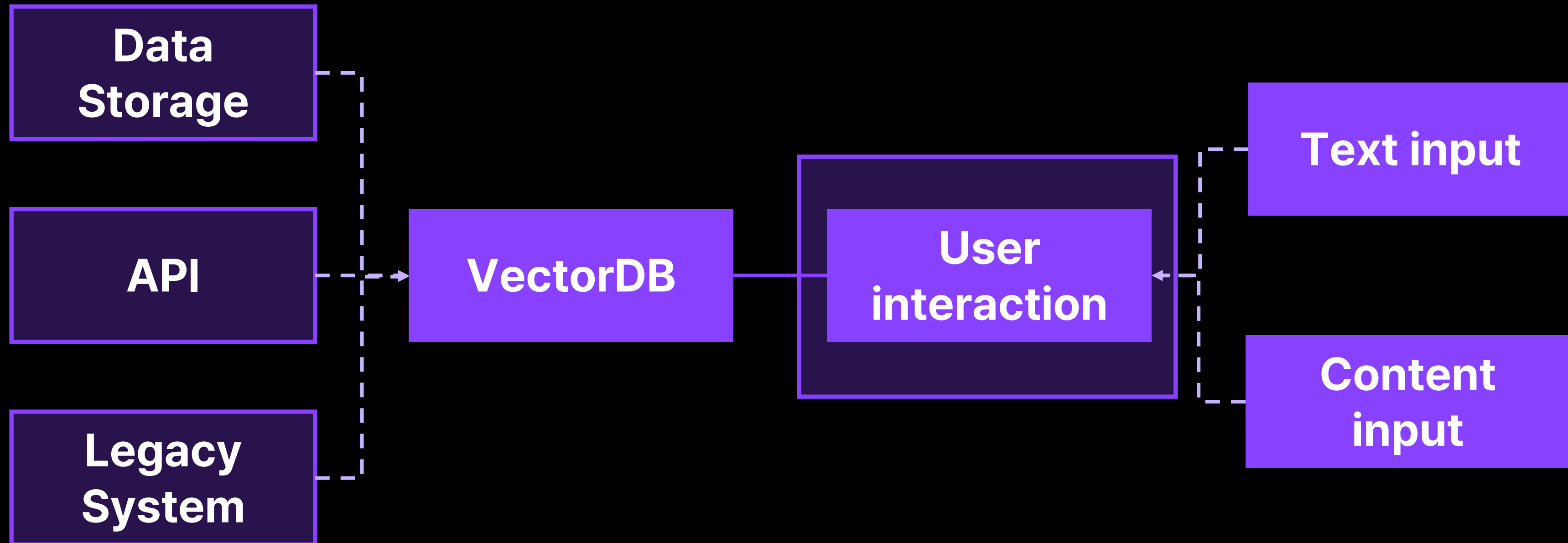


LLM을 활용한 시스템의 일반적인 아키텍처

LangChain과
LangGraph

01 LangChain

1. LLM을 활용한 시스템
의 일반적인 아키텍처



데이터를 수집할 수 있는
data source와 연결

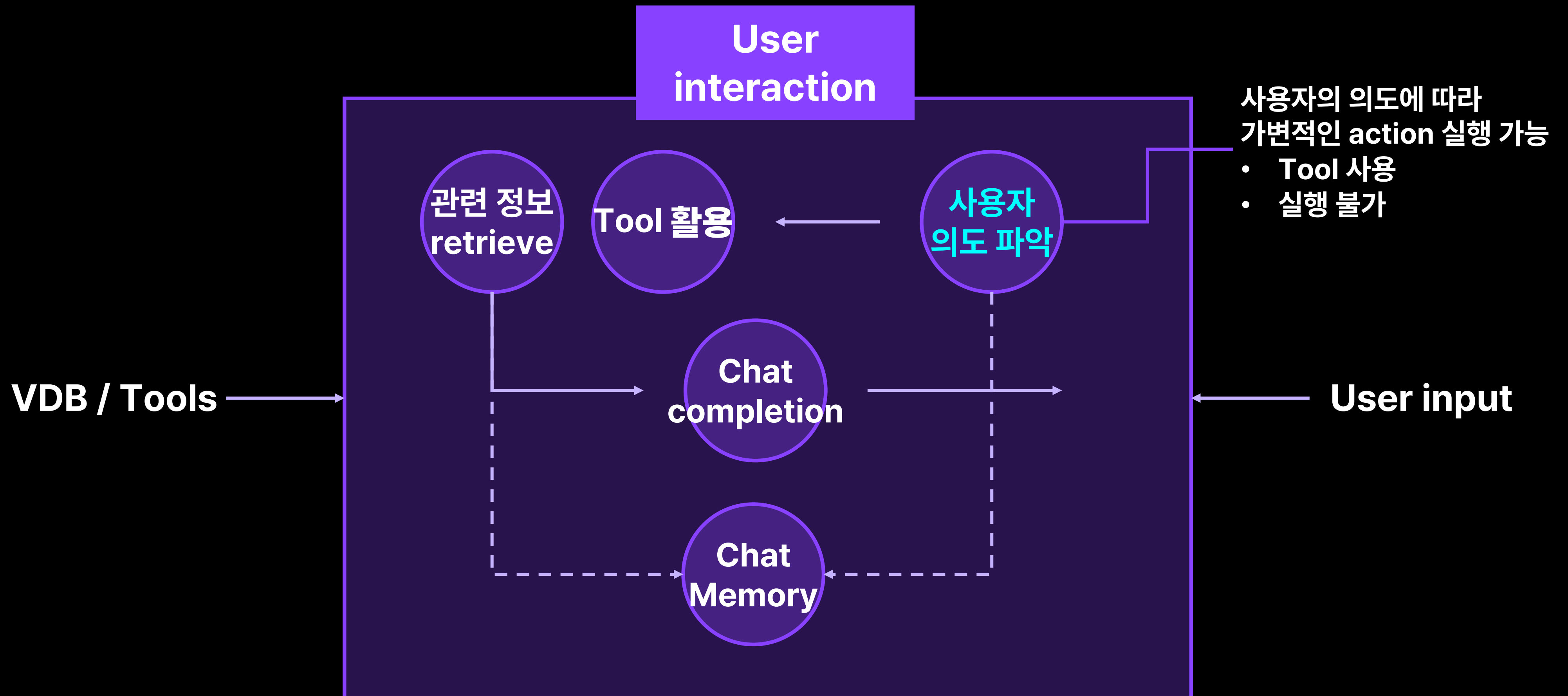
- 기존 legacy data
- API 또는 실시간 data

Data를 VectorDB에 저장

- Embedding vector
- Meta data

사용자 인풋 처리

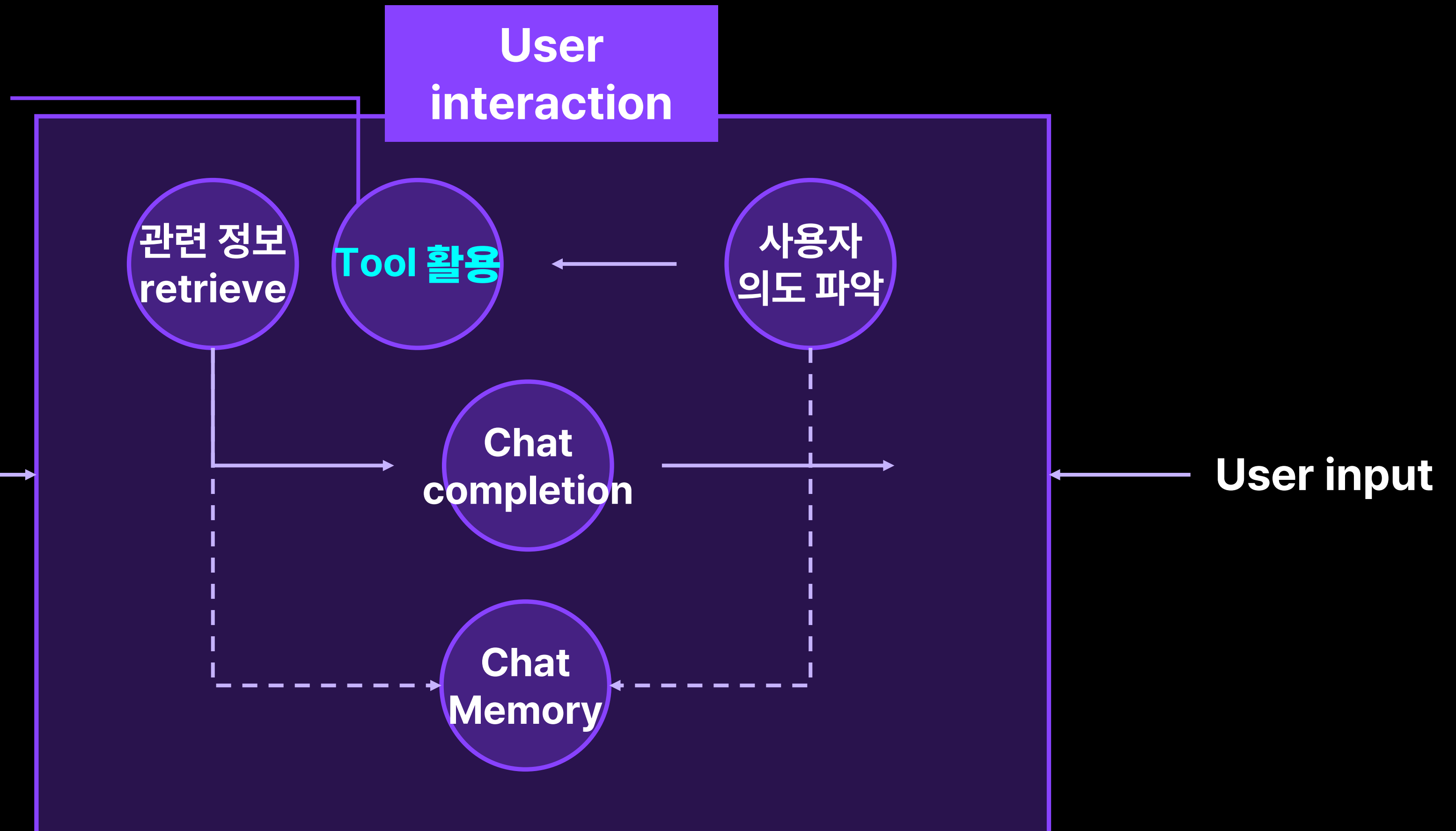
- 시스템의 포맷에 맞는 형태로 재구성
- 시스템 아웃풋을, 사용자와 chat 형식으로 제공



구축된 tool 중 사용자의
의도에 적합한 tool 선택

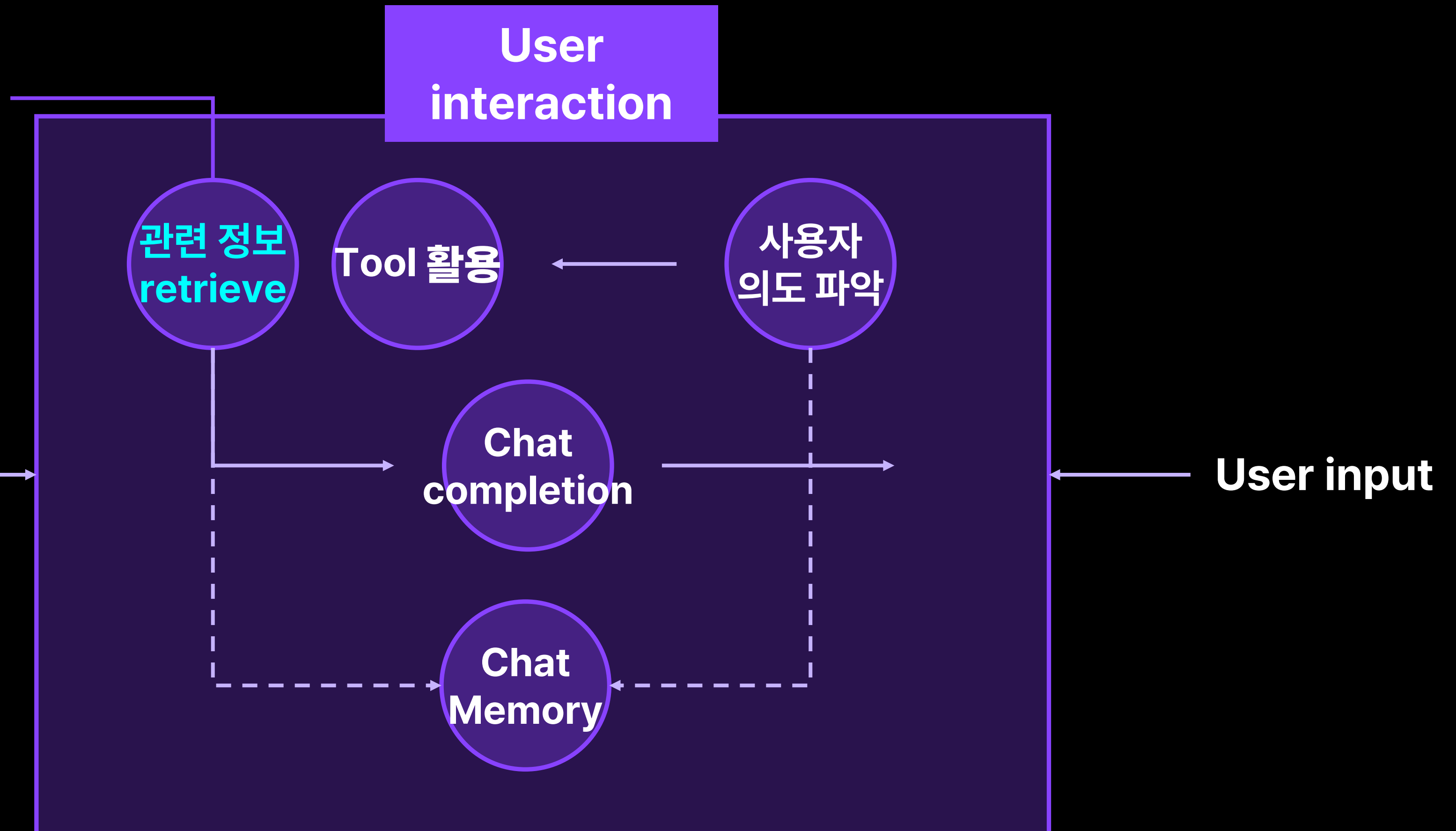
- API call
- Python function

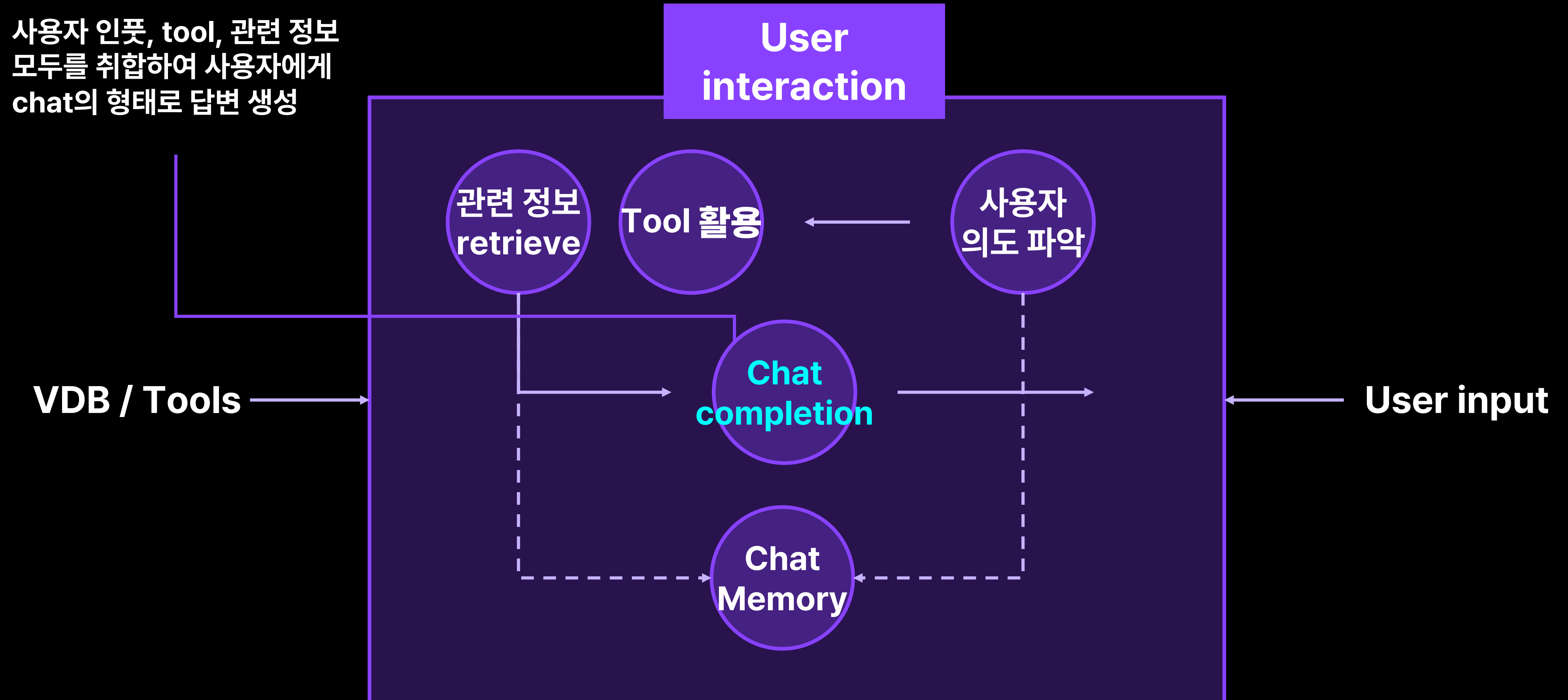
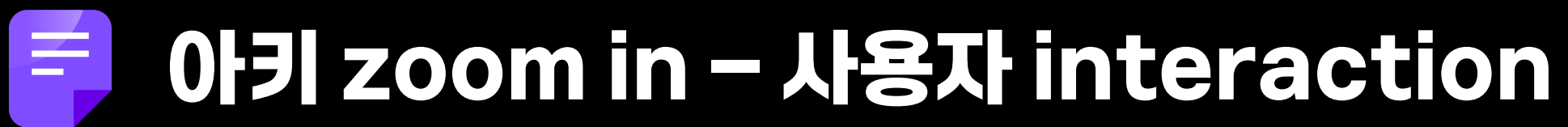
VDB / Tools

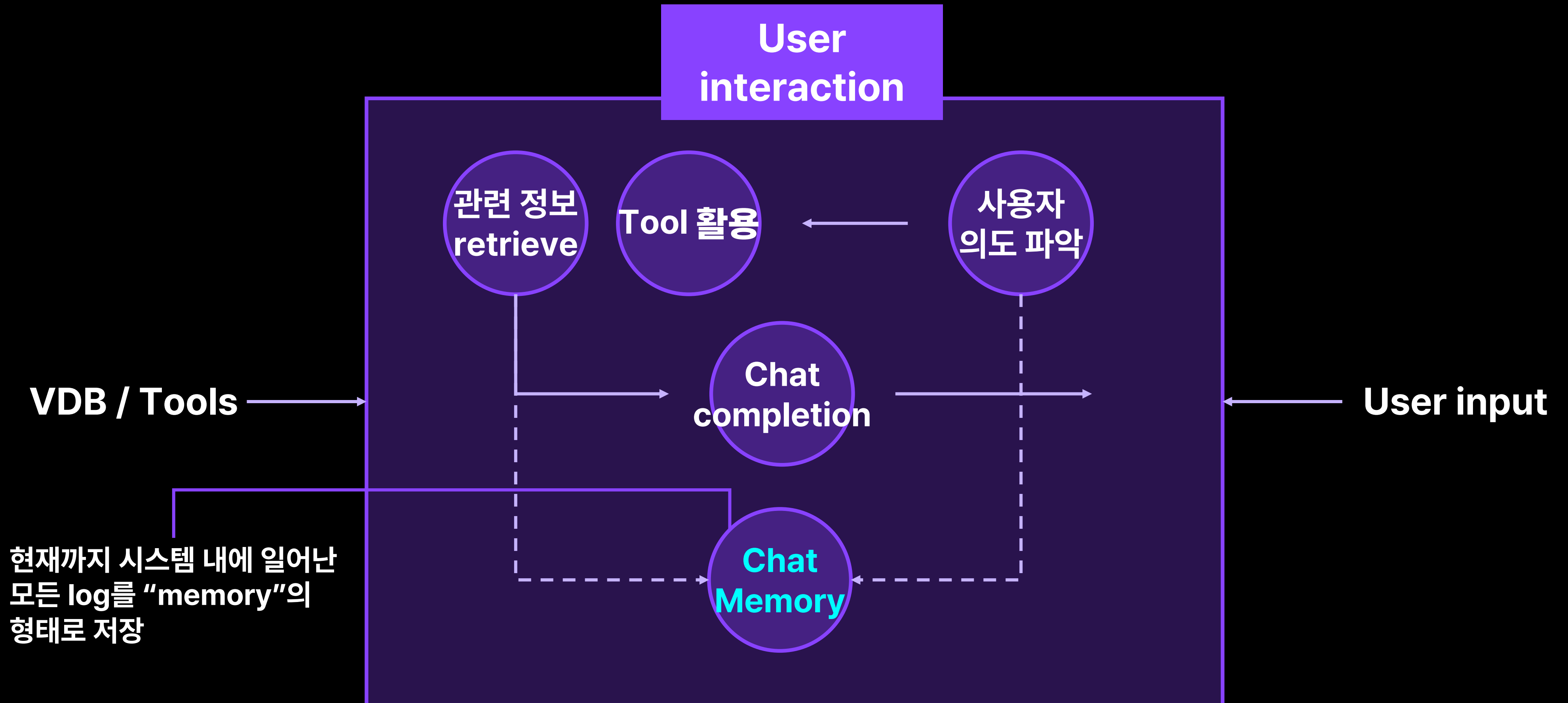


Knowledge base에서
현재 답변에 도움 되는
정보를 "retrieve"하여
인풋에 추가

VDB / Tools







2. LangChain 패키지 구조



LangChain 패키지 구조

LangChain과
LangGraph

01 LangChain

2. LangChain 패키지
구조

LangChain

Langchain의
다양한 컴포넌트를
정의

LangChain
Core

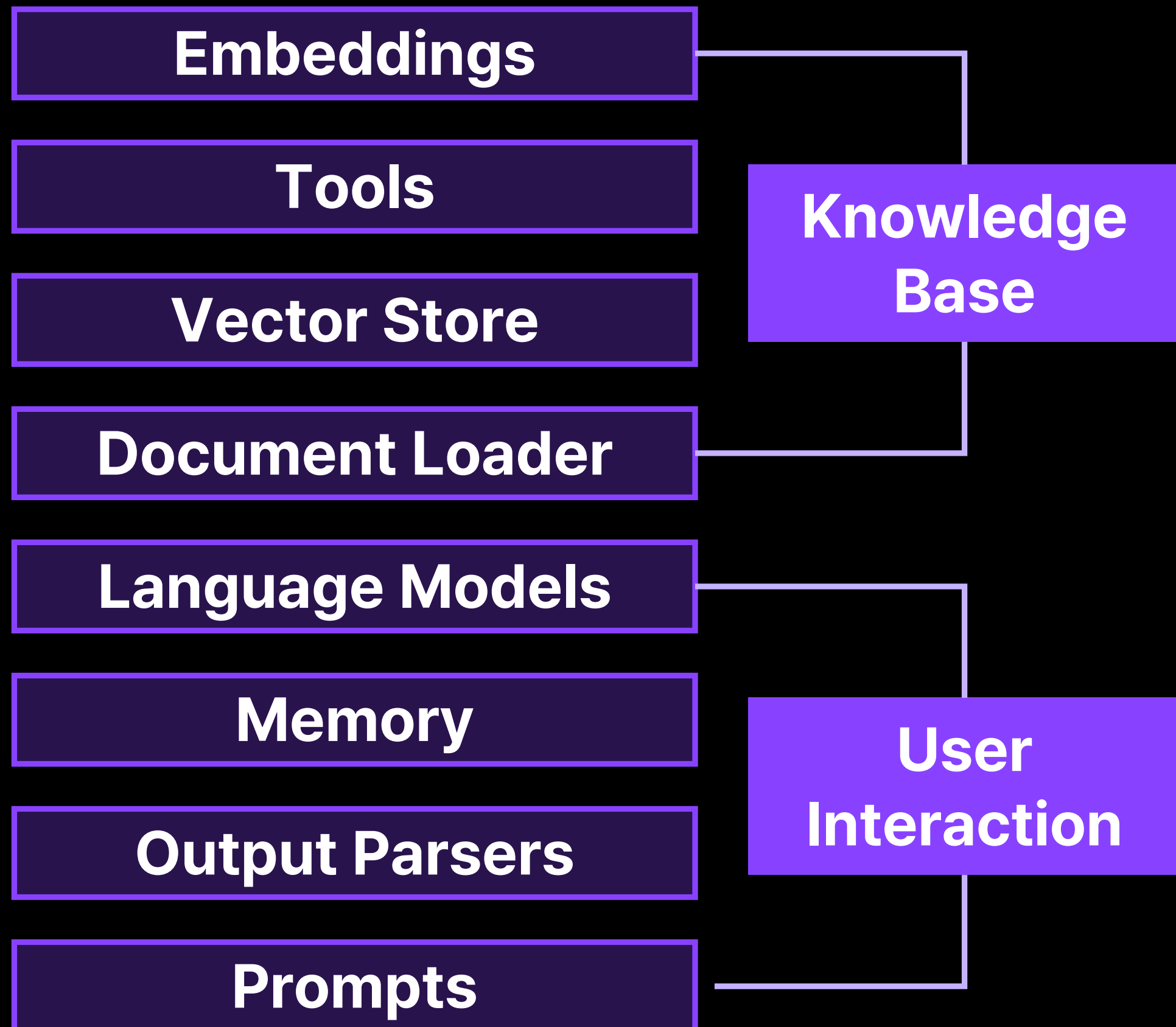
다양한 컴포넌트
구성을 위한 base
package

LangChain
Community

외부 패키지와의
연동을 통해
고도화된 llm 시스템
구축 가능

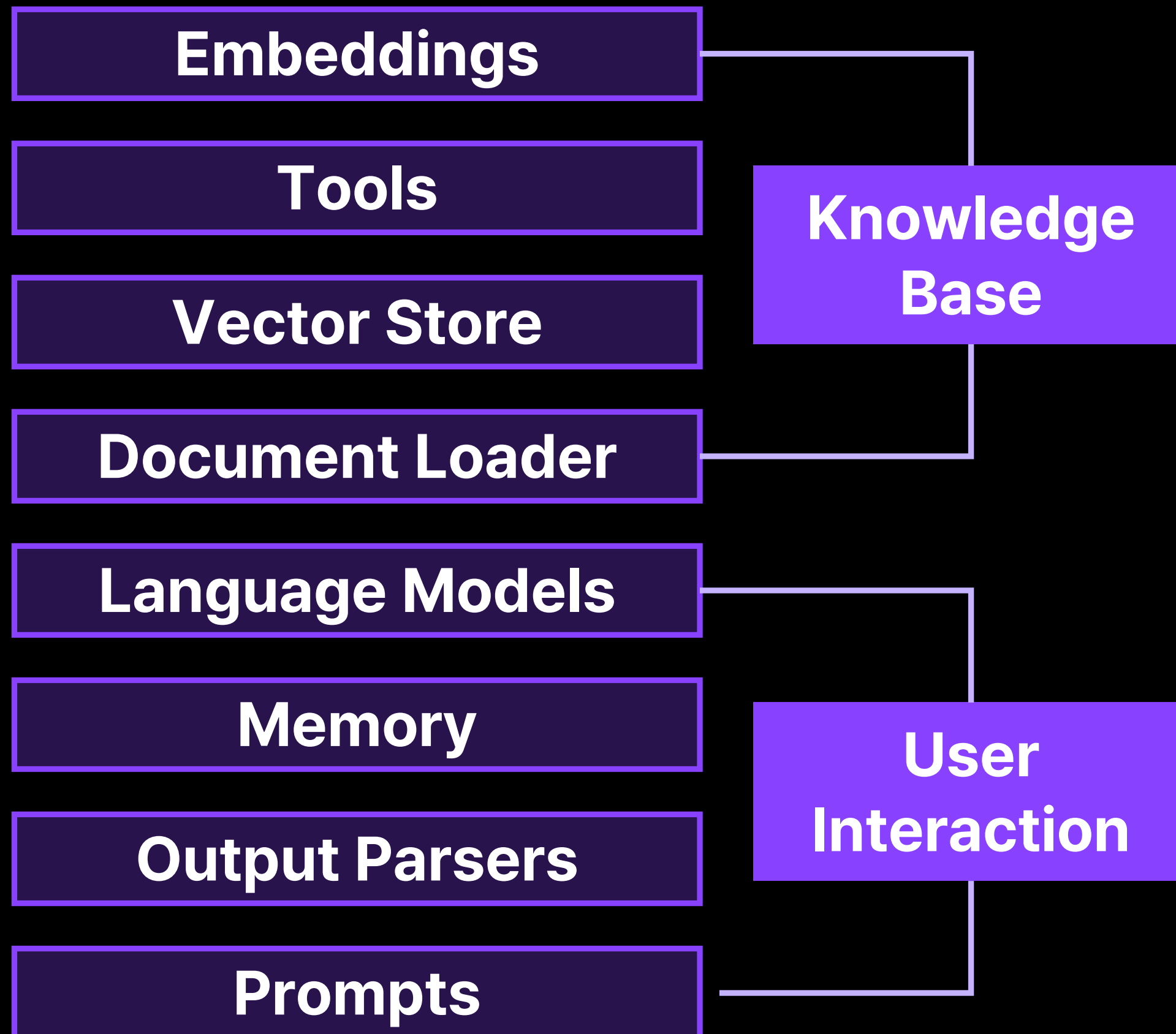
Experimental

초기 개발 단계인
만큼 다양한
실험적인
function들을
추가함



Knowledge base 구축을 위해 필요한 methods

- Embeddings :
텍스트를 vector의 형태로 변환
- Tools :
다양한 function이나 외부 툴과 연동
- Vector Store :
임베딩 벡터를 다른 metadata와 함께
Database에 저장
- Document loader :
다양한 형태의 파일을 LangChain의 인풋으로 변환



사용자 **interaction**을 위해 필요한 **methods**

- Language Models:
GPT 등 가장 핵심이 되는 LLM
- Memory :
사용자와 LLM이 나눈 “대화”의 history를 저장
- Output parsers :
LLM의 아웃풋을 다른 시스템의 input으로
활용 할 수 있도록 특정 형태로 강제함
- Prompts :
LLM의 input

